

Cursor for 2D Animations — Build Plan & Spec (v2)

*Working title: **Manimate** (rename freely). An AI platform that turns a natural-language prompt into a narrated, multi-scene educational explainer video, using Manim as the rendering engine.*

v2 changes: cost & latency engineering woven throughout, an outline-approval gate before generation, an eval/regression harness, multi-frame + best-of-N vision critique, caching and basic sandboxing promoted to MVP, bounded render concurrency, and a dedicated **Bottlenecks** section.

Architecture in one paragraph

The user types a topic or prompt. A **planner** breaks it into a sequence of short, independent **scenes** (this is what dodges the context-window problem — you never generate one giant artifact), and the user **approves or edits that scene outline before anything renders**. Each scene is generated **narration-first**: write the script, then generate Manim code whose animations sync to that script. Every scene passes **two-layer verification** — a free *compile check* (does it render or crash) and a *vision critic* (sample a few rendered frames, have a vision model check overlap / off-screen / intent, then either repair or fall back to best-of-N). A shared **style spec** is injected into every scene so 20 independent generations still look like one coherent film. Verified scenes carry their own synced audio (via `manim-voiceover`), then get **stitched** with FFmpeg into the final video. Everything is **cached by content hash** so unchanged work is never redone. The CLI is the engine; the web UI is a skin on top.

Honest coupling to know up front: because animation timing is derived from narration audio duration, editing narration text means re-rendering that scene. That's inherent, not a bug — we make it cheap with TTS + render caching rather than try to eliminate it.

Core open-source we build on: Manim (Community Edition, version-pinned), `manim-voiceover`, FFmpeg, a vision-capable model for the critic, and any OpenAI-compatible model for generation. The **generator and critic are different, swappable models** (strong coder to write, cheaper vision model to check).

Core design principles

These are the rules that keep cost, latency, and quality under control. Every phase obeys them.

1. **Cheapest checks first.** Free/deterministic checks (compile) run before any paid model or vision call. Vision is the last and most expensive gate, used only on code that

already compiles.

2. **Draft cheap, escalate on failure.** First pass uses a cheap/fast model. Only escalate to a strong model after repeated failure on a given scene.
 3. **Cache everything by content hash.** Same (scene spec + style spec + model) $\Rightarrow$ reuse the existing render. Never re-render or re-generate unchanged work. Cache TTS audio separately.
 4. **Approve structure before spending.** The scene outline is human-editable *before* generation, so you never pay to render 20 wrong scenes.
 5. **Preview fast, finalize slow.** Iterate on low-res quick renders; produce high-res only on final approval.
 6. **Bound the fan-out.** Scenes generate in parallel, but through a worker pool with a sane concurrency cap — not unbounded — to avoid rate limits and simultaneous cost spikes.
-

Time-phased roadmap

Estimates assume **solo, part-time**. Roughly halve if full-time. Each phase ends in something demoable.

Phase 0 — Spike: prove the core loop (~1 week)

Kill the environment risk early; Manim + LaTeX + FFmpeg setup is the real pain, not the AI.

- Dockerized env with Manim CE (**pinned to a specific version**) + LaTeX + FFmpeg.
- **Run all model-generated code inside that container with no network and non-root** — basic sandbox from day one, costs nothing.
- OpenAI-compatible LLM client; model swappable via `(base_url) / (api_key) / (model)`.
- One CLI script: hardcoded prompt $\rightarrow$ LLM $\rightarrow$ Manim code $\rightarrow$ render $\rightarrow$ MP4.
- **Demo:** one rendered scene from one prompt. Ugly is fine.

Phase 1 — Reliable single scene (~2 weeks)

Where the product's quality is actually won.

- Strong system prompt encoding Manim CE conventions (no CE/GL mixing, no deprecated calls), tied to the pinned version.
- **Compile-repair loop:** on crash, feed the *trimmed* traceback back, retry up to N times (don't resend everything — control cost).
- **Vision-critic loop:** sample multiple frames $\rightarrow$ vision model checks overlap / off-screen / intent $\rightarrow$ repair. **Cap iterations; on non-convergence fall back to best-of-N** (generate several candidates, render, keep the highest-scoring).

- **Tiered models:** cheap model first, escalate to strong only after repeated failure.
- **Eval harness:** a small fixed set of prompts with expected outcomes, re-run on every prompt/model change to catch regressions. Without this you're tuning blind.
- Content-hash caching in place from here.
- Scene spec schema (JSON) locked.
- **Demo:** `generate-scene "explain a hash map collision"` → a clean, non-broken scene, and a regression run that proves a change helped.

Phase 2 — Narrated scene (~1 week)

- Integrate `manim-voiceover`; narration-first generation synced via `with self.voiceover(...)` blocks and bookmarks.
- TTS: free gTTS to start, OpenAI/Azure for quality. Auto-subtitles on. **Cache generated audio.**
- Acknowledge the re-render-on-narration-change tax; caching keeps it cheap.
- **Demo:** the scene explains itself out loud, synced.

Phase 3 — Multi-scene video + consistency (~2 weeks)

- **Planner:** topic → outline → ordered scene specs. This is a quality single-point-of-failure, so:
- **Outline-approval gate:** show the scene list, let the user edit/reorder/cut **before** generating any scene.
- **Style spec:** a compact JSON design system (palette, fonts, object shapes, layout rules) injected into every scene's prompt for visual consistency.
- Parallel scene generation **through a bounded worker pool.**
- FFmpeg concatenation into one continuous video.
- **Demo:** `generate-video "how TCP works"` → approve the outline → a coherent, narrated, multi-scene explainer.

Phase 4 — Editing & human control (~2-3 weeks)

- Regenerate a single scene in isolation (cache makes the rest instant).
- Reorder / add / delete scenes; edit narration + re-time.
- Per-scene tweak prompts ("move the cache box left", "slow this down").
- Project persistence: a video = its scene DAG + specs + cached renders.
- Per-scene version history + rollback to a previous good render.
- *(Optional)* **Cross-scene continuity:** pass a scene's ending state as the next scene's starting context, for diagrams built up incrementally. The style spec handles *look*; this

handles *narrative* continuity.

- **Demo:** fix scene 7, re-stitch in seconds, ship.

Phase 5 — Web UI (~3-4 weeks)

The "website" from the slide — but **ship a thin cut first** so momentum doesn't die in a timeline-editor swamp.

- **Thin v1:** prompt box → editable outline → linear scene list → per-scene preview + regenerate. No fancy NLE yet.
- **Preview vs final:** low-res quick renders in the editor; high-res only on export.
- Backend job queue + **hardened** sandboxed render workers (the day-one container isolation, now with strict resource caps and ephemeral FS for multi-user).
- Stream scene-by-scene so the user reviews scene 1 while scene 8 renders.
- Render caching skips unchanged scenes.
- **Demo:** a non-coder makes a narrated explainer end-to-end in the browser.

Phase 6 — Polish, moat, V2 (ongoing)

- Domain templates/themes (CS, ML, math, systems); branding.
- Reusable component/asset library.
- Record-your-own-voice swap; voice cloning; multi-language via translation.
- RAG over a curated library of good Manim examples to cut hallucination and token cost; optionally a fine-tuned model.
- Sharing, gallery, collaboration, export options.

Reality check: Phases 0-3 (~6 weeks part-time) give you the headline demo — *type a topic, approve the outline, get a narrated multi-scene video*. That's the portfolio milestone. Phases 4-6 turn it from a demo into a product.

Bottlenecks & how we handle them

The honest list of what will bite, and the mitigation now baked into the plan.

Bottleneck	Why it bites	How we handle it
Token/cost blowup	20 scenes × (generate → repair → vision → regenerate) = 100+ calls, vision being the pricey ones	Free compile check before any vision call; cheap model first, escalate on failure; cache by content hash; outline approval avoids rendering wrong scenes
Render latency	Manim is CPU-bound, seconds-minutes/scene; a full video feels slow, worse in a web spinner	Low-res preview vs high-res final; render only changed scenes; stream scene-by-scene; bounded parallel rendering
Blind generation (the core one)	Model writes positioning code with no eyes → overlaps, off-screen, bad timing	Two-layer verification; vision critic samples multiple frames (not one) to catch mid-animation problems
Critic non-convergence	Critic keeps flagging, model keeps failing → infinite loop	Hard iteration cap, then fall back to best-of-N parallel candidates and pick the highest-scoring
Bad scene decomposition	One wrong outline ⇒ everything downstream is polished garbage, fully paid for	Outline-approval gate before any scene renders
Narration editing tax	Animation timing derives from audio duration, so text edits force a re-render	Accept the coupling; make it cheap via TTS + render caching
Cross-scene continuity	Pure scene independence breaks diagrams built up across scenes	Optional persisted end-state handed to the next scene (Phase 4)
Dependency/version drift	Training data mixes Manim versions + ManimGL → silent breakage	Pin a specific Manim CE version; encode its conventions in the system prompt
Arbitrary code execution	Running model-written Python can touch files/network	Locked-down container (no net, non-root) from day one ; strict caps + ephemeral FS at multi-user
Web-UI scope creep	A timeline NLE is a huge frontend that can stall the	Ship a thin linear v1; defer the real editor

Bottleneck	Why it bites	How we handle it
	project	
Flying blind on changes	Prompt tweaks may secretly regress quality	A small fixed eval set re-run on every change

Full feature catalog

Grouped by subsystem. **[MVP]** = needed for the core experience; **[Later]** = V2+.

1. Generation core

- **[MVP]** Natural-language prompt → Manim code
- **[MVP]** Narration-first script generation
- **[MVP]** Scene decomposition / planner (topic → outline → specs)
- **[MVP]** Outline-approval gate (edit scenes before generating)
- **[MVP]** Shared style/object design-spec (cross-scene consistency)
- **[MVP]** Tiered model strategy (cheap draft → strong on failure)
- **[Later]** Domain templates/presets (CS, ML, math, systems)
- **[Later]** Image / PDF / arXiv input → explainer

2. Reliability & verification (*the part that separates you from the existing demos*)

- **[MVP]** Compile-check + auto-repair loop (trimmed traceback fed back)
- **[MVP]** Vision-critic loop, multi-frame sampling
- **[MVP]** Best-of-N fallback on non-convergence + hard iteration caps
- **[MVP]** Eval/regression harness
- **[MVP]** Manim API guardrails tied to a pinned version
- **[Later]** Per-scene quality score surfaced to the user

3. Audio & narration

- **[MVP]** TTS integration (gTTS free → OpenAI/Azure)
- **[MVP]** Auto-synced narration (`manim-voiceover`)
- **[MVP]** Auto subtitles/captions
- **[MVP]** TTS audio caching
- **[Later]** Per-word / bookmark fine sync
- **[Later]** Swap AI voice → recorded human voice

- [Later] Multi-language (translation + localized TTS)

4. Composition & editing

- [MVP] Scene-DAG project model
- [MVP] Bounded parallel scene rendering
- [MVP] FFmpeg stitching/concatenation
- [MVP] Regenerate a single scene in isolation
- [Later] Reorder / add / delete scenes
- [Later] Edit narration text + re-time
- [Later] Per-scene tweak prompts
- [Later] Project save/load/persistence
- [Later] Per-scene version history + rollback
- [Later] Cross-scene continuity (persisted end-state)

5. Rendering & infrastructure

- [MVP] Content-hash render cache (cross-cutting, designed in early)
- [MVP] Basic sandbox (no-net, non-root container) from day one
- [MVP] Low-res preview vs high-res final
- [Later] Hardened sandbox (resource caps, ephemeral FS) for multi-user
- [Later] Job queue + async status
- [Later] Quality/resolution settings

6. Interface

- [MVP] CLI core (the engine everything wraps)
- [Later] Thin web UI: prompt → outline → scene list → preview → regenerate
- [Later] Streaming scene-by-scene progress
- [Later] Full timeline editor
- [Later] Export/download (MP4 ± burned-in subtitles)

7. Power features / moat

- [Later] Style themes & branding
- [Later] Reusable component/asset library
- [Later] RAG over curated Manim examples
- [Later] Optional fine-tuned Manim model
- [Later] Sharing / collaboration / public gallery

Real-life use cases

Concrete "who uses it and what they actually make":

1. **YouTube explainer creators** — a solo creator makes a 3Blue1Brown-style video on "how attention works in transformers" without months of Manim. Type the concept, get the video.
2. **University lecturers** — a professor generates animated visuals (eigenvectors rotating, a Fourier transform decomposing a wave, a B-tree rebalancing) instead of static slides, narrated to match their notes.
3. **Online course creators (Udemy / Coursera / bootcamps)** — a whole library of lesson videos at scale, kept visually uniform across 40 lessons by the shared style spec.
4. **Developer relations / technical marketing** — "how our consensus protocol reaches agreement," "how a request flows through our caching layer" (the exact client → server → cache video you prototyped). Launch videos, docs companions.
5. **Students making revision videos** — turn a hard concept into an animation to actually understand and retain it; the act of generating the visualization is the studying.
6. **Researchers explaining papers** — an arXiv paper or a key figure becomes an animated walkthrough for a talk or a "paper explained" post, with no motion designer.
7. **Interview / DSA prep content** — animate algorithms and data-structure operations step by step (two-sum, Dijkstra, a sliding window) where seeing the pointers move is the whole value.
8. **K-12 / EdTech teachers** — animated math and science for younger students (geometry proofs, projectile motion, fractions), with friendly narration and multi-language audio.
9. **Corporate L&D / onboarding** — animated process flows and "how our data pipeline works" so new engineers grok the architecture faster than reading a wiki.
10. **Conference speakers & pitches** — clean animated diagrams and sequences for a talk or investor pitch instead of fighting Keynote to fake motion.

The one risk to keep front-of-mind

Generating Manim code is close to commoditized — several open-source projects already do it. The hard, **unsolved** part is making the output reliably *look* right and wrapping it in a coherent, affordable, low-latency multi-scene product. Everything in the Bottlenecks table is in service of that. Your two load-bearing features remain the **vision-critic loop** and the

style spec — they're why your version looks finished when the others look like demos. The cost/latency discipline above is what makes it usable past a toy.